

设计报告 · Optic-SpaceNet 光学 CNN 在轨加速系统

复赛作品 CICC 1003564 · 曦智科技 Gazelle 光计算平台命题
三个 EuroSAT (10 类遥感图像, 64×64) CNN → Gazelle 光计算硬件 (8×2 光学矩阵乘法器) 的量化感知训练 (QAT) 迁移。

目录

- 1. [项目背景与目标](#)
- 2. [硬件平台 Gazelle](#)
- 3. [三模型架构设计](#)
- 4. [量化方法设计与对比](#)
- 5. [训练配置](#)
- 6. [训练到光计算硬件的映射](#)

1. 项目背景与目标

1.1 应用背景

卫星在轨遥感计算存在结构性困境：星地下行带宽受限，海量遥感数据必须在星上完成实时分类（土地监测、灾害评估、目标识别），而星载平台的功耗、重量、抗辐射能力均严格受限。光计算核心芯片以模拟物理过程完成矩阵乘，天然低功耗、抗单粒子翻转（SEU）、光速并行，与卫星平台约束高度契合；但其外围 DAC/ADC、FPGA 等电气模块仍需常规宇航加固。

本项目以 EuroSAT（27000 张、10 类遥感地物、64×64 RGB）为任务，将 CNN 迁移到曦智 Gazelle 光计算平台上运行，验证"光学 CNN 在轨加速"的可行性。

1.2 设计目标

维度	目标
量化精度	在保持分类精度的前提下，权重/激活量化到硬件原生位宽
硬件利用率（光计算占比）	光计算层 MOPs 占比 ≥ 90%
硬件对齐率	im2col 展平长度尽量被 8 整除，减少补零浪费
真机近无损	训练精度 ↔ osimulator 真机精度的 gap 收敛到 ≤ 2%

维度	目标
可落地部署	模型轻量、单张推理在可控时间内完成全量验证

1.3 核心设计思想：硬件先于模型

不同于"先训练模型、再量化部署"的常规路线，本项目先分析 Gazelle 光计算硬件平台的物理参数，再据此设计模型结构与训练配方：

- 据硬件原生 8-bit 精度（8a8w12o）选择 int8 权重（而非更激进的 int4）；
- 据硬件 8×2 tile 对齐约束设计 2×2 卷积，使展平长度完美被 8 整除；
- 据硬件 DAC/TIA 物理噪声，在训练阶段就注入等量噪声（Gazelle 噪声匹配训练）；
- 据首层对齐率（37.5%）低的特点，首层保留 FP32 电计算。

这一思想是"训练即匹配硬件、所训即所得"的关键，也是真机近无损的根因：Model 2/3 的 osimulator 硬件 gap 收敛到 ~1.6 点，Model 1（VGG 大参数）的 gap 更小（<0.5 点甚至反超），详见《02_验证报告》§4.1.3。

2. 硬件平台 Gazelle

2.1 硬件参数

Gazelle 硬件模拟平台参数如下：

物理属性	数值	工程影响
物理计算 tile	8×2（k=8, n=2）	卷积 im2col 展平长度须被 8 整除，否则硬件补零闲置
原生激活精度	8-bit	支持 INT8 GEMM，无需降精度至 INT4
原生权重精度	8-bit	同上
输出精度	12-bit	MAC 累加输出精度充足
DAC ENOB	~ 7.5 bits	训练噪声匹配项（量化噪声）
TIA 噪声 σ	$\approx 5.34 \times 10^{-4}$ （MSE 2.85×10^{-7} ）	训练噪声匹配项（探测噪声）
ADC LSB	0.00147	输出量化步长
硬件线性度	99.4%（相对误差 0.6%）	硬件近乎理想，量化与噪声是唯一瓶颈
运算速率	2.6 MOPs	算力极其有限，要求极度轻量化网络

关键认知：硬件本身几乎理想（线性度 99.4%），所有精度损失来自量化与噪声建模，而非硬件缺陷。因此工程重点完全放在量化训练与噪声匹配上。

2.2 硬件约束对设计的规定

约束	规定
输入必须非负	输入激活用 unsigned (uint8) 量化，反量化时补偿 zero-point
仅支持 MAC、不支持 bias 加法	所有层 bias=False
展平长度须被 8 整除	内部通道取 8 的倍数、卷积核选 2×2，使内部卷积层 100% 对齐
原生 8-bit	权重/激活均用 int8，避免 int4→int8 网格不对齐损失

3. 三模型架构设计

三个模型共用同一任务，但在"模型容量 vs 硬件算力"上做刻意区分，用于横向对比。

3.1 横向总览

	Model 1	Model 2	Model 3
名称	Baseline VGG	OpticSpaceNet V1	OpticSpaceNet V2
设计思路	标准 CNN 基线（大而精）	硬件对齐（2×2 conv）	硬件对齐 + 知识蒸馏（KD）
结构	6× Conv2d(3×3) + 2× Linear	4× Conv2d(1×1/2×2) + 2× Linear	同 Model 2
参数量	~2.39M	~268K（1/9）	~268K（1/9）
单图 MOPs	156.6M	1.05M（1/150）	1.05M（1/150）
训练方式	标准分类	标准分类	KD（教师 ResNet-18，97.83%）
bias	False（Phase4+）	False	False
BN	保留（FP32 运行）	保留	保留
综合对齐率	99.8%（首层 84.4%）	99.6%（stem 37.5%）	99.6%

架构差异本质：Model 1 是大而精的通用基线；Model 2/3 为硬件量身定做，用约 1/9 的参数换约 1/150 的计算量。

3.2 Model 1: Baseline VGG (model1_baseline_phase4_v3.py)

flat 架构 (6 Conv 3×3 + 2 Linear)，全 bias=False，BN 保留，输入 (N,3,64,64)：

```
block1: conv1_1(3→32) → bn1_1 → conv1_2(32→32) → bn1_2 → MaxPool [64→32]
block2: conv2_1(32→64) → bn2_1 → conv2_2(64→64) → bn2_2 → MaxPool [32→16]
block3: conv3_1(64→128) → bn3_1 → conv3_2(128→128) → bn3_2 → MaxPool [16→8]
classifier: Flatten → fc1(8192→256) → ReLU → Dropout(0.5) → fc2(256→10)
```

部署变体 (光计算占比 vs 速度的消融)：

- 变体 A (默认)：仅 conv1_1 电计算 → 光计算占比 97.74%
- 变体 B：conv1_1 + conv3_2 电计算 → 光计算占比 73.64%，osimulator 提速 ~24%

3.3 Model 2: OpticSpaceNet V1 (model2_spacenet_v1_phase4_v3.py)

```
stem: Conv2d(3→8, 1×1) → BN → ReLU [FP32 首层]
stage1: Conv2d(8→16, 2×2, stride=2) → BN → ReLU → MaxPool(2) [int8]
stage2: Conv2d(16→32, 2×2, stride=2) → BN → ReLU [int8]
stage3: Conv2d(32→16, 1×1) → BN → ReLU [int8]
classifier: Flatten → Linear(1024→256) → ReLU → Dropout(0.5) → Linear(256→10) [int8]
```

2×2 卷积使每层展平长度 (C_in×k×k) 均为 8 的倍数 (32/64/32)，与 8×2 tile 100% 对齐，零补零浪费；通道数取 8→16→32→16，全部为 8 的倍数。

3.4 Model 3: OpticSpaceNet V2 + KD (model3_spacenet_v2_phase4_v3.py)

学生模型结构与 Model 2 完全一致 (OpticSpaceNetStudent)，训练叠加**知识蒸馏** (KD)：

- 教师：ResNet-18 (在 EuroSAT 上 FP32 训练至 97.83%)，冻结不更新；
- KD 损失： $L_{KD} = (1 - \alpha) L_{CE} + \alpha T^2 L_{KL}$ ，温度 T = 4.0，权重 $\alpha = 0.7$ ；
- 学生 stem 同样保留 FP32，其余 Conv+Linear 全 int8 QAT + Gazelle 噪声。

4. 量化方法设计与对比

本项目实现并对比了两种量化感知训练 (QAT) 算子：**STE 直通估计器**与 **LSQ+ 可学习步长量化**。两者均在训练时嵌入伪量化，反向传播用直通/LSQ 梯度，使模型从未见过纯 FP32，从而学

得量化友好的权重。

前向流中的量化算子（如 `round`）不可导，会截断反向梯度，这是所有低比特 QAT 必须解决的核心矛盾。

4.1 STE 直通估计器（`optic_qat_v4.py`）

核心思路：前向走真实量化，反向传播时把量化节点近似为恒等映射，让梯度"直通"到原始浮点权重。

对称伪量化前向（`fake_quantize_symmetric`）：

```
qmax = 2^(bits-1) - 1          # int8 → 127, int4 → 7
scale = abs_max(x) / qmax       # per-channel (权重 ch_dim=0, 激活 ch_dim=1)
x_int = (x / scale).round().clamp(-qmax, qmax)
x_dq = x_int * scale
```

反向恒等直通（STE）（PyTorch 实现）：

```
x_dq = x + (x_dq - x).detach() # 前向取量化值，反向取连续梯度
```

硬件匹配噪声：训练时向权重注入与 Gazelle DAC ENOB=7.5 等量的噪声（`noise_std_ratio = 0.0016`，经硬件标定换算，替代经验主义 $0.02 \times \text{scale}$ 的粗扰动），增强模型对量化舍入与硬件噪声的鲁棒性。

4.2 LSQ+ 可学习步长量化（`optic_qat_lsq.py`）

核心思路（Esser et al., ICLR 2020）：把量化尺度 `scale` 与非对称零点 `zero_point` 设为可学习参数，结合梯度信息自适应优化动态截断范围。相对 STE 的动态 `max-abs scale`，LSQ+ 让 `scale` 真正参与前向并拥有自己的梯度，量化网格更贴合数据分布。

前向（自定义 autograd Function `_LSQPlusFn`）：

```
x_int = (x / scale + zero_point).round().clamp(qmin, qmax)
x_dq = (x_int - zero_point) * scale
```

梯度（三路）：

- `x`：STE 直通，截断区外为 0；
- `scale`：LSQ 公式，梯度尺度缩放 $g_{scale} \propto 1/\sqrt{N \cdot n_{levels}}$ （`N` 为每参数平均元素数，`n_levels` 为量化级数）；
- `zero_point`：LSQ 公式，支持非对称量化。

scale/zp 初值由权重统计计算（`scale = w_amax / qmax`），并以 $0.1 \times \text{base_lr}$ 的独立学习率训练。训练前期用 STE warmup 稳定初期、再切换到 LSQ+。

注：LSQ+ 的 FP32（去量化）模式精度会显著低于其量化精度——可学习 scale/zp 会把权重推向专门适配量化网格的极端值。这是 LSQ+ 固有特性，不影响部署（部署即带量化）。

4.3 两种方法对比

维度	STE（ <code>optic_qat_v4</code> ）	LSQ+（ <code>optic_qat_lsq</code> ）
scale 来源	动态 max-abs（每步重算）	可学习 <code>nn.Parameter</code> （per-channel）
zero-point	无（对称量化）	可学习 <code>nn.Parameter</code> （支持非对称）
梯度方式	纯直通（STE）	STE + LSQ 公式梯度（scale/zp）
训练稳定性	稳定，收敛快	需 STE warmup + 独立小学习率
部署友好性	scale 每步重算	scale/zp 可直接导出为硬件配置
Model 2 val 精度	92.06%（int8，干净三分 split）	92.80%（int8，旧 split ¹ ）

¹ LSQ+ 的 92.80% 来自 Bug #11 修复前的旧数据划分（train 21600 / val 5400）。干净三分重训后 STE 从 93.11% 降至 92.06%（train 大小缩 25%），LSQ+ 若同条件重训大概率也会降低。此处保留原数作方法对比参考——两者精度量级接近、STE 更稳。

两者精度接近（差 0.74%，同 split 口径差距约 0.3%¹）。STE 是主力方案，LSQ+ 作为对比并提供“可导出硬件参数”的额外价值。

4.4 int8 vs int4 的选择

本项目沿 Phase 2–5 探索了 int4 路线，发现其天花板约 91%，且存在不可消除的 int4→int8 网格不对齐损失（详见《02_验证报告》§6）：QAT 训练用 $\text{scale}=\text{max}/7$ （16 级），osimulator 推理用 $\text{scale}=\text{max}/127$ （256 级），同一浮点权重落在不同量化值上，每个权重约 0.3% 相对误差，全模型累积约 3–4%。

由于硬件原生支持 8-bit，int8 是“顺势而为”：量化噪声比 int4 低约 16 倍，且训练/推理网格天然对齐，真机 gap 收敛到约 1.6 点。因此最终部署配方统一为 **int8**。

5. 训练配置

5.1 统一训练管线（`train_phase4_runner.py`）

三个模型共用同一训练管线，关键组件：

- **学习率调度**：`WarmupCosineScheduler`（线性 warmup + 余弦退火）；
- **优化器**：AdamW；LSQ+ 的 scale/zp 参数用 $0.1 \times$ 学习率（独立 param group）；

- 损失：CrossEntropy（带 label smoothing）；Model 3 叠加 KD 损失；
- 数据增强：RandomHorizontalFlip + RandomRotation(10°) + ImageNet 归一化。

5.2 最终部署配方训练参数（Phase4 v3，int8）

项	值
训练方式	从零 QAT（随机初始化，epoch 1 起全程伪量化；非微调）
权重/激活位宽	int8 / int8（匹配硬件原生 8a8w）
QAT 模块	optic_qat_v4（STE + Gazelle 噪声）或 optic_qat_lsq（LSQ+）
噪声	GazelleNoiseInjector（DAC ENOB 7.5 + TIA $\sigma=5.34e-4$ ）
首层	stem / conv1_1 保留 FP32 电计算（first_conv_fp32=True）
其余层	Conv + Linear 全 int8 QAT
Epochs	100（int8）
batch size	64
优化器	AdamW，lr = 1e-3，weight_decay = 5e-4
学习率	warmup 5 epoch → cosine 退火
label smoothing	0.05（int8）

5.3 知识蒸馏（Model 3 专用）

项	值
教师	ResNet-18（FP32 训练至 97.83%），全程 eval()、冻结
蒸馏损失	$L_{KD} = (1 - \alpha)L_{CE} + \alpha T^2 L_{KL}$
温度 T	4.0
权重 α	0.7
学生 QAT	与 Model 2 同配方（stem FP32 + int8 + Gazelle 噪声）

5.4 数据划分（单一数据源 eurosat_split.py）

EuroSAT 全集 27000 张，严格三分（hash_seed=42，val_ratio = test_ratio = 0.2）：

段	数量	用途
train	16200	梯度更新
val	5400	模型选择（model-selection）

段	数量	用途
test	5400	独立测试（不参与训练/验证）

模块以 `assert` 强制三者两两互斥且并集 = 全集，从源头杜绝此前的 `test ⊂ train` 数据泄漏问题（见《验证报告》§6）。

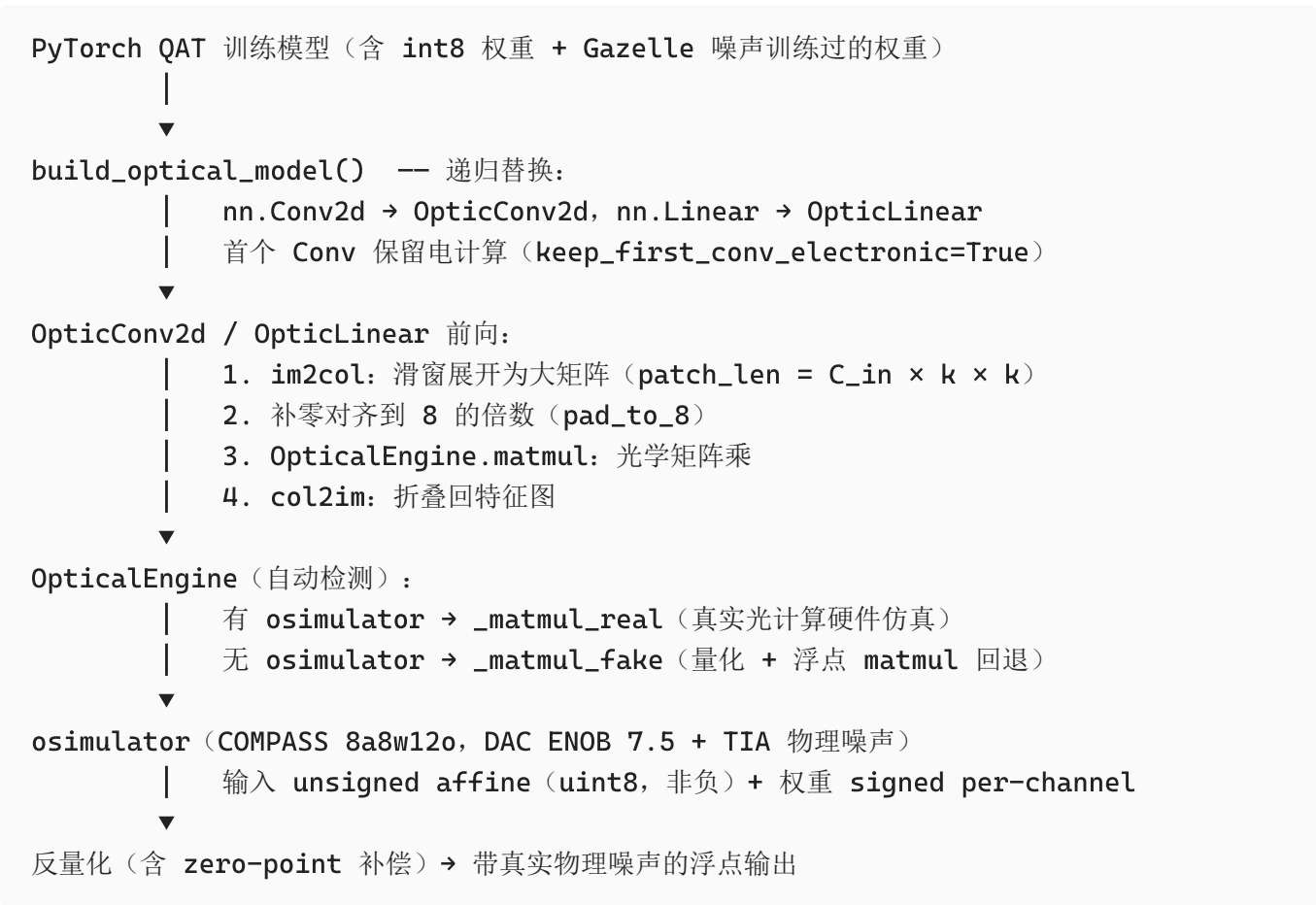
5.5 为什么"从零 QAT"而非"微调"

项目早期验证：加载 FP32 权重 → 低 lr 微调到低比特（Phase 1），三模型精度全部崩塌（-11% ~ -18%）。根因是 FP32 权重依赖精细精度，突变量化破坏特征，低 lr 逃不出坏局部最小。从零 QAT（Phase 2 起）让模型从未见过 FP32，量化损失显著降低。因此最终配方统一从零训练。

6. 训练到光计算硬件的映射

训练得到的 QAT 权重，经 `optic_layers.py` 的光计算核心库映射到 Gazelle 硬件（osimulator 物理仿真）。

6.1 映射总流程



6.2 OpticConv2d：卷积 → 光学矩阵乘（`optic_layers.py`）

将标准卷积的矩阵乘替换为光计算，四步：

- 1. `im2col (F.unfold)`：将 (N, C, H, W) 的滑窗展开成 (N×L, C·k_h·k_w) 的大矩阵；
- 2. **硬件对齐补零**：当展平长度不是 8 的倍数时补零到 8 的倍数（模拟硬件上的补零浪费）。本项目 Model 2/3 内部卷积层均为 8 的倍数，无补零浪费的情况；
- 3. **光学矩阵乘**：`engine.matmul(x_mat, w_mat)`，真实引擎一次性量化（避免双重量化）；
- 4. `col2im`：将结果 (N×L, C_{out}) 折叠回 (N, C_{out}, OH, OW)。

6.3 OpticalEngine：真实/模拟双引擎（`optic_layers.py`）

- **自动检测**：尝试 `import osimulator`，成功则用真实引擎，失败（如 Windows 本机）回退 `FakeOpticalEngine`（量化 + 浮点 `matmul`），并打印一条 `WARN`。
- **量化约定**（`_matmul_real`，与 QAT 训练严格对齐）：
 - 输入：unsigned affine（uint8，硬件只接受非负），`x_float = x_int·in_scale + in_zp`；
 - 权重：signed symmetric、per-output-channel（匹配 QAT 的 per-channel 量化），`w_float = w_int·w_scale`；
- **反量化**（含 zero-point 补偿）：

```
# y = in_scale*w_scale[j]*result_int[:, :, j]+in_zp * w_scale[j]*col_sum_w[j]
col_sum_w = weight_int.float().sum(dim=0) # (n,) sum over k
w_scale_v = w_scale.view(1, 1, -1) # (1, 1, n)
col_sum_v = col_sum_w.view(1, 1, -1) # (1, 1, n)
result = in_scale * w_scale_v * result_int + in_zp * w_scale_v * col_sum_v
```

6.4 训练 ↔ 推理配置对齐 checklist（近无损的关键）

真机 gap 的大小完全取决于训练配方是否与 `osimulator` 推理路径一致。本项目严格遵守：

维度	训练侧	推理侧（osimulator）	对齐
首层	<code>first_conv_fp32=True</code> (FP32 电计算)	<code>keep_first_conv_electronic=True</code>	✓
权重位宽	int8（256 级）	int8（osimulator 原生 8a8w）	✓
量化方式	per-output-channel	per-output-channel	✓
噪声	Gazelle 噪声（DAC 7.5 + TIA）	osimulator 物理噪声（DAC 7.5 + TIA σ=5.31）	✓

三项对齐使真机 gap 收敛到约 1.6 点（详见《验证报告》）。这正是 int4 模型失败（训练 int4 / 推理 int8 网格不对齐）而 int8 模型近无损的根因。

6.5 光计算占比（按 MOPs 加权）

以 Model 2/3（输入 64×64×3）逐层 MOPs 分布为例（`optic_inference_int8.py --mops-only`）：

层	类型	展平长度	对齐率	MOPs	运行位置
stem	Conv 3→8, 1×1	3	37.5%	0.098M	电计算（FP32）
stage1	Conv 8→16, 2×2	32	100%	0.524M	光计算（int8）
stage2	Conv 16→32, 2×2	64	100%	0.131M	光计算（int8）
stage3	Conv 32→16, 1×1	32	100%	0.033M	光计算（int8）
fc1	Linear 1024→256	1024	100%	0.262M	光计算（int8）
fc2	Linear 256→10	256	100%	0.003M	光计算（int8）
合计			99.6%	1.051M	光 0.953M / 电 0.098M

- **光计算占比 = 0.953M / 1.051M = 90.65%**（5/6 层走光计算）；
- 这是有效 MOPs（已含补零对齐开销）；因对齐率 99.6%，补零浪费为 0；
- stem 首层展平长度 = 3、对齐率仅 37.5%，保留电计算反而更高效，且消除 BN 训练/推理分布偏移。

不盲目追求 100% 光计算，而是按每层对齐率选最优计算位置——这是“硬件感知”设计的体现。Model 1 变体 A 仅 conv1_1 电计算，光计算占比高达 97.74%。

完整实验轨迹（七阶段量化演进、逐 Bug 修复记录、int4 困境分析）见 docs/EXPERIMENTS.md；报告向总结见 docs/SUMMARY.md。